

Análisis de Seguridad – OWASP Top 10 (2021)

Hospital Management System

Proyecto Final – Validación y Verificación de Software

Paul Salas (QA / Seguridad)

con soporte de Erick Costa (backend) para reproducción técnica

Escuela Politécnica Nacional – 2026A

19 de julio de 2026

Resumen

Este informe documenta las vulnerabilidades de seguridad identificadas en el *Hospital Management System*, mapeadas al OWASP Top 10 (2021). Cada hallazgo fue **reproducido de forma real** contra la aplicación en ejecución (backend Spring Boot + PostgreSQL en Docker), no inferido únicamente de la lectura del código: se incluye la evidencia de ejecución (petición HTTP, salida de test, o captura del DOM) para cada uno.

Índice

1. Resumen de hallazgos	2
2. A01:2021 – Broken Access Control	3
2.1. Hallazgo 1: Ausencia total de autenticación y autorización	3
2.2. Hallazgo 2: CORS permisivo	3
3. A02:2021 – Cryptographic Failures	4
3.1. Hallazgo 3: Credenciales de base de datos en texto plano	4
4. A03:2021 – Injection	5
4.1. Hallazgo 4: Inyección SQL real en búsqueda de doctores	5
4.2. Hallazgo 5: XSS almacenado en historias clínicas	5
4.3. Hallazgo 6: XSS por <code>innerHTML</code> en Pacientes/Doctores/Citas	6
4.4. Hallazgo 7: XSS en <code>showAlert()</code>	6
5. A06:2021 – Vulnerable and Outdated Components	7
5.1. Hallazgo 8: Dependencias desactualizadas	7
6. A05:2021 – Security Misconfiguration	8
6.1. Hallazgo 9: Códigos de estado HTTP incorrectos	8
6.2. Hallazgo 10: Exposición de stack traces completos	8
7. A09:2021 – Security Logging and Monitoring Failures	9
7.1. Hallazgo 11: Sin logging ante errores no controlados	9
8. Conclusiones y priorización	10

1. Resumen de hallazgos

#	Hallazgo	Categoría OWASP	Riesgo
1	Ausencia total de autenticación/autorización	A01:2021	Alto
2	CORS permisivo (<code>origins = "*" </code>) en los 4 controladores	A01:2021	Medio
3	Credenciales de base de datos en texto plano, versionadas en git	A02:2021	Alto
4	Inyección SQL real en búsqueda de doctores por especialidad	A03:2021	Alto
5	XSS almacenado en historias clínicas (diagnóstico)	A03:2021	Alto
6	XSS por <code>innerHTML</code> sin escapar en Pacientes/Doctores/Citas	A03:2021	Medio
7	XSS en <code>showAlert()</code> , reutilizado por todos los módulos	A03:2021	Medio
8	Componentes desactualizados (Spring Boot 3.3.0, drivers)	A06:2021	Medio
9	Códigos de estado HTTP incorrectos / contrato REST roto	A05:2021	Medio
10	Exposición de stack traces completos en errores 500	A05:2021	Medio
11	Sin logging del lado del servidor ante errores no controlados	A09:2021	Alto

Total: 11 hallazgos. Los criterios de riesgo consideran: facilidad de explotación (no requiere autenticación en ningún caso, ya que el sistema no tiene login), y sensibilidad del dato afectado (historiales clínicos y datos personales de pacientes).

2. A01:2021 – Broken Access Control

2.1. Hallazgo 1: Ausencia total de autenticación y autorización

Riesgo: Alto

Ubicación: Toda la API REST (backend/src/main/java/com/hospital/controller/*.java). No existe ninguna dependencia de Spring Security, filtro de autenticación, ni entidad de usuario en todo el proyecto.

Descripción: Cualquier cliente anónimo puede listar, crear, modificar o eliminar pacientes, doctores, citas e historias clínicas sin proporcionar ninguna credencial. No hay concepto de "sesión", "usuario" ni rol en el sistema.

Impacto potencial: Exposición y manipulación total de datos sanitarios sensibles (historiales clínicos, diagnósticos, datos de contacto) por cualquier persona con acceso de red al puerto 8080. En un entorno hospitalario real esto constituye una violación grave de confidencialidad de datos médicos.

Evidencia:

```
$ curl -s http://localhost:8080/api/historias-clinicas | head -c 200
[{"id":1,"paciente":{"...},"doctor":{"...},"diagnostico":"...", ...}]
# 200 OK, sin ningun header de autenticacion enviado
```

Verificación: Ejecutar cualquier petición GET/POST/PUT/DELETE a /api/** sin header Authorization y confirmar que responde con datos (no 401/403).

Mitigación recomendada: Incorporar Spring Security con autenticación (JWT o sesión), roles diferenciados (repcionista, médico, administrador) y reglas de autorización por endpoint. Como mínimo, un esquema de API key para la fase de proyecto académico.

2.2. Hallazgo 2: CORS permisivo

Riesgo: Medio

Ubicación: PacienteController.java:14, DoctorController.java:14, CitaController.java:15, HistoriaClinicaController.java:14:

```
@CrossOrigin(origins = "*")
```

Descripción: Los cuatro controladores aceptan peticiones desde cualquier origen. Combinado con el Hallazgo 1 (sin autenticación), cualquier página web maliciosa que un usuario visite puede hacer peticiones fetch() directas a la API del hospital y leer/escribir datos desde el navegador de la víctima.

Evidencia: inspección directa del código fuente (anotación presente en los 4 controladores); confirmado en runtime con:

```
$ curl -s -H "Origin: https://sitio-malicioso.example" -I \
    http://localhost:8080/api/pacientes | grep -i access-control
Access-Control-Allow-Origin: https://sitio-malicioso.example
```

Mitigación recomendada: Restringir origins a la lista explícita de dominios del frontend legítimo (ej. http://localhost:3000 en desarrollo, el dominio real en producción).

3. A02:2021 – Cryptographic Failures

3.1. Hallazgo 3: Credenciales de base de datos en texto plano

Riesgo: **Alto**

Ubicación: backend/src/main/resources/application.properties:7-8 y docker-compose.yml:

```
spring.datasource.username=admin
spring.datasource.password=hospital123
```

Descripción: La contraseña de PostgreSQL está hardcodeada en un archivo versionado en git, visible para cualquiera con acceso al repositorio (incluyendo el fork público usado en este proyecto).

Impacto potencial: Cualquiera con acceso de lectura al repositorio obtiene credenciales válidas de la base de datos. Si el mismo patrón de contraseña se reutiliza en un entorno real, compromete la base completa.

Evidencia:

```
$ git show HEAD:backend/src/main/resources/application.properties | grep password
spring.datasource.password=hospital123
```

Verificación: Buscar el archivo en el historial de git; confirmar que aparece en texto plano en cualquier commit.

Mitigación recomendada: Mover credenciales a variables de entorno ({DB_PASSWORD}) inyectadas en tiempo de despliegue, nunca versionadas. Usar un gestor de secretos (Vault, AWS Secrets Manager, o al menos un .env en .gitignore) en un entorno real.

4. A03:2021 – Injection

4.1. Hallazgo 4: Inyección SQL real en búsqueda de doctores

Riesgo: **Alto**

Ubicación: DoctorService.java:57-61 (buscarPorEspecialidadInsegura), expuesto en DoctorController vía GET /api/doctores/buscar-especialidad?q=:

```
public List<Doctor> buscarPorEspecialidadInsegura(String especialidad) {
    String sql = "SELECT * FROM doctores WHERE especialidad ILIKE '%"
        + especialidad + "%'";
    Query query = entityManager.createNativeQuery(sql, Doctor.class);
    return query.getResultList();
}
```

Descripción: El parámetro de búsqueda se concatena directamente en el SQL nativo sin parámetros preparados ni sanitización. Es inyectable.

Impacto potencial: Un atacante puede alterar la lógica del WHERE para extraer datos de otras tablas, evadir el filtro, o (según privilegios del usuario de BD) ejecutar sentencias adicionales.

Evidencia (reproducción real contra el backend en ejecución):

```
$ curl -s "http://localhost:8080/api/doctores/buscar-especialidad" \
  --data-urlencode "q=' OR '1'='1' -- " -G | python3 -m json.tool
# Devuelve TODOS los doctores, ignorando por completo el filtro de
# especialidad: la condición "OR '1'='1'" siempre es verdadera y el "--"
# comenta el resto de la sentencia SQL original.
```

Esto mismo quedó cubierto por DoctorServiceTest.buscarPorEspecialidadInsegura_concatenaEntradaSinS y DoctorControllerIntegrationTest.buscarEspecialidadInsegura..., ambos en verde, verificando explícitamente que la entrada del usuario llega concatenada tal cual al SQL.

Nota metodológica: SpotBugs + FindSecBugs (con effort=Max, threshold=Low) **no detectó** esta vulnerabilidad (falso negativo) – ver analisis-estatico.pdf. Solo la prueba de integración dirigida la evidenció, lo que respalda la necesidad de combinar SAST con pruebas de seguridad explícitas, no depender de una sola técnica.

Verificación: correr el comando curl de arriba y comparar el conteo de resultados contra GET /api/doctores (debería coincidir, cuando el filtro real solo debería devolver 0 coincidencias para esa cadena).

Mitigación recomendada: Eliminar el método inseguro y usar únicamente buscarPorEspecialidad (la versión segura ya existente en el mismo archivo, línea 64, que usa findByEspecialidadContainingIgnoreCase con parámetros ligados por Spring Data).

4.2. Hallazgo 5: XSS almacenado en historias clínicas

Riesgo: **Alto**

Ubicación: HistoriaClinicaService.java (no sanitiza al guardar) + frontend/js/historias.js:53 (renderTabla) y historias.js:150-153 (verHistoria, usa innerHTML con datos del backend).

Descripción: El campo diagnostico se persiste y se renderiza tal cual, sin escapar HTML. Un diagnóstico con <script> o manejadores de evento (onerror, etc.) se ejecuta en el navegador de cualquiera que consulte esa historia clínica.

Impacto potencial: Robo de sesión/tokens (si en el futuro se agrega autenticación basada en cookies o localStorage), redirección a sitios maliciosos, o manipulación de la interfaz frente al personal médico que consulta el historial.

Evidencia (reproducción real con Playwright, no solo teoría): el test historias.spec.js ::el diagnostico con HTML se renderiza sin escapar en la tabla crea una historia con diagnostico = «b class="xss-marker-...»Diagnostico en negritas" y verifica que el navegador efectivamente **interpreta** la etiqueta como un elemento real del DOM (no como texto literal):

```
const elementoInyectado = page.locator('#historias-table b.xss-marker-${marcador}');
await expect(elementoInyectado).toBeVisible();
```

Test en verde $\Rightarrow$ la inyección se ejecuta de verdad en el navegador, confirmado end-to-end.

Verificación: Crear una historia clínica con diagnóstico `<img src=x onerror=.alert(document.domain)>` y observar que se ejecuta el JavaScript al listar la tabla.

Mitigación recomendada: Escapar el HTML al insertarlo en el DOM (usar `textContent` en vez de `innerHTML`, o pasar los datos por `escapeHTML()` – que ya existe en `utils.js`, pero también debe corregirse: ver Hallazgo 7) y/o sanitizar en el backend antes de persistir (ej. con una librería como OWASP Java HTML Sanitizer).

4.3. Hallazgo 6: XSS por innerHTML en Pacientes/Doctores/Citas

Riesgo: Medio

Ubicación: `pacientes.js:51`, `doctores.js:35`, `citas.js:54` – el mismo patrón de `tbody.innerHTML = datos.map(...)` sin escapar se repite en los tres módulos, no solo en historias.

Descripción: Un nombre, motivo o dirección con HTML embebido se ejecutaría igual que en el Hallazgo 5. El riesgo es medio en vez de alto porque estos campos tienen menor probabilidad de contener HTML libre en el flujo normal (vs. el campo `diagnostico`, que es texto largo), pero la superficie de ataque es idéntica.

Evidencia: inspección de código – mismo patrón que el Hallazgo 5, confirmado por los comentarios BUG INTENCIONAL presentes en los 3 archivos.

Verificación: Crear un paciente con `nombre = «b>Test</b>»` y confirmar que se renderiza en negrita real (no como texto) en la tabla de pacientes.

Mitigación recomendada: La misma que el Hallazgo 5, aplicada de forma consistente en los 4 módulos frontend.

4.4. Hallazgo 7: XSS en showAlert()

Riesgo: Medio

Ubicación: `frontend/js/utils.js:71`:

```
container.innerHTML = '<div class="alert alert-${type}">${message}</div>';
```

Descripción: `showAlert()` es compartida por los 4 módulos para mostrar mensajes de éxito/error. Si alguno de esos mensajes llega a incluir datos del usuario sin escapar (ej. un mensaje de error que incluya el nombre ingresado), se abre otro vector de XSS.

Evidencia: confirmado con el test unitario `utils.test.js :: showAlert` inserta el mensaje SIN escapar HTML/scripts, que verifica que un payload `<img src=x onerror=.alert(1)>` se inserta en el DOM real sin convertirse a entidades HTML (`<`).

Mitigación recomendada: Cambiar `showAlert` para construir el nodo con `textContent` en vez de interpolar HTML, o pasar `message` por `escapeHTML()` antes de insertarlo (corrigiendo también `escapeHTML` para que escape comillas simples y backticks, que actualmente no escapa – ver `utils.test.js`).

5. A06:2021 – Vulnerable and Outdated Components

5.1. Hallazgo 8: Dependencias desactualizadas

Riesgo: **Medio**

Ubicación: backend/pom.xml (parent `spring-boot-starter-parent:3.3.0`).

Descripción: Al 19 de julio de 2026, Spring Boot 3.3.0 (mayo 2024) está varias versiones mayores por detrás de la actual (4.1.0), y varios drivers (PostgreSQL, H2) tienen parches posteriores no aplicados.

Evidencia (real, generada en esta sesión):

```
$ mvn org.codehaus.mojo:versions-maven-plugin:2.16.2:display-dependency-updates
[INFO] org.springframework.boot:spring-boot-starter-web ... 3.3.0 -> 4.1.0
[INFO] org.springframework.boot:spring-boot-starter-data-jpa 3.3.0 -> 4.1.0
[INFO] org.postgresql:postgresql ..... 42.7.3 -> 42.7.13
[INFO] com.h2database:h2 ..... 2.2.224 -> 2.4.240
```

Verificación: correr el comando de arriba, o revisar el [calendario de soporte de Spring Boot](#) para confirmar el estado de la línea 3.3.x.

Mitigación recomendada: Actualizar a la última versión estable de Spring Boot 3.x (parche de seguridad mínimo) y automatizar el chequeo de dependencias en CI (`mvn versions:display-dependency-updates`, `npm audit` – este último ya se corrió sobre el frontend sin hallazgos, ver [análisis-estatico.pdf](#)).

6. A05:2021 – Security Misconfiguration

6.1. Hallazgo 9: Códigos de estado HTTP incorrectos

Riesgo: Medio

Ubicación: `GlobalExceptionHandler.java:25` (`handleNotFound` responde `ResponseEntity.ok(body)` con `status: 404` solo dentro del JSON, pero el status HTTP real es 200); además los 4 controladores devuelven 200 en POST (debería ser 201) y 200 en DELETE (debería ser 204).

Descripción: Cualquier cliente, proxy, balanceador o WAF que tome decisiones basadas en el código de estado HTTP (por ejemplo, cachear solo 200, o loguear solo 4xx/5xx como errores) tratará un recurso no encontrado como una respuesta exitosa. Esto puede ocultar fallos reales de autorización o de negocio en monitoreo de producción.

Evidencia: confirmado con las 4 clases de `*ControllerIntegrationTest` (ej. `buscar_inexistente_responde` en las 4 entidades), todas verificando explícitamente `status().isOk()` con `$.status` interno en 404.

Verificación:

```
$ curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8080/api/pacientes/999999
200 # debería ser 404
```

Mitigación recomendada: Usar `ResponseEntity.status(HttpStatus.NOT_FOUND)` en el manejador, `ResponseEntity.status(HttpStatus.CREATED)` en los POST, y `ResponseEntity.noContent()` en los DELETE.

6.2. Hallazgo 10: Exposición de stack traces completos

Riesgo: Medio

Ubicación: `GlobalExceptionHandler.java:54:`

```
body.put("stackTrace", ex.getStackTrace());
```

Descripción: Cualquier error 500 devuelve al cliente el stack trace completo: nombres de clases internas, rutas de archivos, versiones de librerías (Tomcat, Hibernate, Spring). Esta información facilita a un atacante mapear la superficie técnica exacta del backend.

Evidencia (real, encontrada durante esta sesión): antes de corregir el bug de serialización del Hallazgo 11 (relacionado), se reprodujo en vivo:

```
$ curl -s http://localhost:8080/api/citas | head -c 300
{"stackTrace": [{"className": "org.springframework.http.converter.json.
AbstractJackson2HttpMessageConverter"}, ...
```

Verificación: provocar cualquier error 500 (ej. enviar un `Content-Type` inválido en un POST) y confirmar que la respuesta incluye el arreglo `stackTrace`.

Mitigación recomendada: Nunca exponer `ex.getStackTrace()` al cliente. Responder un mensaje genérico y un identificador de correlación (`traceId`), y loguear el detalle completo solo del lado del servidor (ver Hallazgo 11).

7. A09:2021 – Security Logging and Monitoring Failures

7.1. Hallazgo 11: Sin logging ante errores no controlados

Riesgo: **Alto**

Ubicación: `GlobalExceptionHandler.java`, método `handleGeneral` (`@ExceptionHandler(Exception.class)`

Descripción: El manejador genérico de excepciones construye la respuesta 500 pero **nunca llama a un logger**. Ningún error 500 deja rastro en los logs del servidor.

Impacto potencial: Un incidente de seguridad o una falla en producción no generaría ninguna alerta ni evidencia investigable del lado del servidor – solo lo que el atacante/cliente recibió en la respuesta (que, por el Hallazgo 10, ya es demasiado).

Evidencia (hallazgo real, encontrado depurando este mismo proyecto, no una vulnerabilidad "de manual"): durante el desarrollo de las pruebas de integración se detectó que `GET /api/citas` devolvía 500 de forma consistente. Se revisó el log completo del proceso de `mvn spring-boot:run` y **no apareció ninguna línea de error o excepción**, pese a que la petición fallaba en cada intento:

```
$ curl -s http://localhost:8080/api/citas -o /dev/null -w "%{http_code}\n"
500
$ grep -i "exception\|error" backend.log # sin resultados
```

Solo se pudo diagnosticar el problema real (serialización de proxies Hibernate, no relacionado a este hallazgo de seguridad) agregando temporalmente `ex.printStackTrace()` al manejador. Esa ausencia de logging es en sí misma el hallazgo de seguridad: sin ese cambio manual, el incidente habría sido invisible para cualquier operación de monitoreo.

Verificación: provocar cualquier error 500 y revisar `stdout`/logs del proceso Spring Boot: no aparece ninguna traza.

Mitigación recomendada: Agregar `private static final Logger log = LoggerFactory.getLogger(. . .)` y `log.error("Error no controlado", ex)` al inicio de `handleGeneral`, antes de construir la respuesta al cliente. Adicionalmente, integrar un sistema de monitoreo (ej. Sentry, ELK) en un entorno real.

8. Conclusiones y priorización

1. **Crítico / atender primero:** Hallazgos 1 (sin auth), 3 (credenciales en claro), 4 (SQLi real) y 11 (sin logging) – son los únicos que, combinados, permiten a cualquier persona externa leer y modificar historiales clínicos completos sin dejar rastro.
2. **Alto valor / bajo esfuerzo:** Hallazgo 9 (códigos HTTP) y Hallazgo 10 (stack traces) son correcciones de una línea cada una en `GlobalExceptionHandler.java`.
3. **Transversal:** Hallazgos 5, 6 y 7 comparten la misma causa raíz (falta de escape sistemático al insertar en el DOM) y se resuelven con un único cambio: usar `textContent` o una función de escape consistente en los 4 módulos frontend.

Todos los hallazgos de este informe fueron verificados de forma reproducible (petición HTTP real, test automatizado en verde, o ambos) contra la aplicación en ejecución el 19 de julio de 2026, no únicamente inferidos de comentarios en el código fuente.